

A Safeguarded Hybrid Classical-Quantum Solver for Distributed Order Management

Validated distributed order allocation with exact recourse and bounded quantum proposals

Andrei Pomorov

August 2026

Abstract

Distributed order management (DOM) must select a distribution center and ship date for each order while sharing time-phased inventory and operational capacity across many orders. The decision is combinatorial, but the delivered plan must also satisfy exact case balance, load cohesion, eligibility, and cumulative available-to-promise (ATP) constraints. We present a reproducible solver hierarchy that combines deterministic greedy construction, exact fixed-assignment recourse, adaptive mixed-integer large-neighborhood search (LNS), and an experimental sampler-assisted neighborhood search. The sampler may be classical, simulated annealing, local Quantum Approximate Optimization Algorithm (QAOA), or an IBM quantum processor; it can only propose bounded assignment moves. Deterministic one-hot repair, exact mixed-integer linear programming (MILP) recourse, and an independent validator retain authority over feasibility.

The reviewed study contains 247 aggregate rows across 14 experiment families. Every row passed the independent validator, with zero maximum residual for demand balance, integrality, cumulative inventory, and enabled capacity constraints. On a common 20-assignment-group real-data subset, default routing captured 61.39% of requested merchandise value in the business objective, while greedy routing reached 64.90%; exact polishing added only 0.004 percentage points. Polished greedy, exact LNS, and the full exact MILP agreed to the displayed precision, with runtimes of 2.86, 7.00, and 2.76 seconds respectively. A deliberately coordinated synthetic control exposed a greedy trap: exact LNS, exact MILP, simulated annealing, and local QAOA improved the objective by 197.2 synthetic units. A 26-row hardware archive on IBM [ibm_marrakesh](#) showed that all final solutions were valid after recourse, but deeper QAOA sharply reduced native one-hot feasibility. These findings support a practical classical deployment path and a falsifiable quantum research program; they do not establish quantum advantage.

Keywords: distributed order management; available to promise; mixed-integer linear programming; large-neighborhood search; QUBO; QAOA; hybrid quantum-classical optimization; feasibility repair

1. Introduction and related literature

An order allocator must decide more than where an order can ship. It must decide which eligible origin and planned-goods-issue (PGI) date should serve the order, how much of each stock-keeping unit (SKU) can be filled, and whether a reassignment is valuable enough to justify transport and operational disruption. These decisions interact through inventory checkpoints, dock limits, shared loads, and service penalties. A locally attractive reassignment may consume inventory needed by a more valuable order, and line-by-line routing can violate load cohesion. DOM is consequently a mixed discrete-continuous optimization problem in which feasibility is operationally more important than the score reported by any one solver.

The classical foundation is well established. Available-to-promise allocation has been formulated as mixed-integer optimization to coordinate order acceptance and allocation [1]. Order-fulfillment models have jointly considered inventory, delivery commitments, and production or distribution decisions [2], while modern e-fulfillment research continues to use MILP to represent allocation, service, and logistics trade-offs [3]. These formulations provide strong constraints and optimality bounds, but a full model can become expensive as every order-candidate-SKU combination is expanded.

Decomposition and neighborhood search address that scaling problem. Shaw introduced large-neighborhood search as repeated exact or heuristic re-optimization of a destroy and repair neighborhood [4]. Local branching defines a MILP neighborhood around an incumbent [5], and relaxation-induced neighborhood search uses information from the linear relaxation to focus a subproblem [6]. The common principle is useful here: keep the valid global incumbent, expose only a bounded set of interacting routing decisions, and solve quantities exactly after the local assignments are chosen.

Quantum optimization introduces a different proposal mechanism. QAOA alternates a problem-cost operator and a mixer to sample low-cost binary states [7]. The Quantum Alternating Operator Ansatz generalized this framework to constraint-preserving mixers and feasible initial states [8]. For one-hot assignment groups, an XY mixer preserves the number of excitations, while deterministic Dicke-state constructions prepare a known feasible superposition [9,10]. Parameter initialization and concentration remain important because variational optimization itself can dominate runtime [11]; warm-start methods and quantum local search therefore place quantum routines inside a classical outer loop rather than asking a processor to solve a large operational model directly [12,13].

Our contribution is an auditable integration of these ideas for the challenge data:

1. a documented column-based MILP with exact demand, eligibility, protected ATP, capacity, diversion, and thresholded-penalty rules;
2. a portable hierarchy from greedy construction to exact LNS, with SciPy/HiGHS as the license-free default and Gurobi as an optional comparison backend;
3. a bounded QUBO proposal layer whose output cannot bypass deterministic repair, exact recourse, or independent validation;
4. a controlled evidence suite spanning common-method comparison, scaling, sensitivity, robustness, ablation, synthetic coordination, and IBM hardware; and
5. a stronger final QAOA implementation with linear W-state preparation, a connected path mixer, reusable optimized parameters, and quality metrics relative to exact and uniform-feasible controls.

The practical conclusion is intentionally conservative. The current solver's primary advantage is the architecture: it is useful before, during, and after any quantum experiment because it always preserves a trusted feasible incumbent.

2. Problem definition and mathematical model

2.1 Data and decision unit

The challenge supplies five runtime tables covering orders, shipping alternatives, time-phased inventory, throughput observations, and remaining dock capacity. The adapter validates required identifiers, dates, domains, and numeric fields before constructing the optimization model. Restricted identifiers and raw commercial totals are not reproduced in this report.

Orders on the same source load form an **assignment group** and must select one common distribution-center/PGI option. A **candidate** is an eligible distribution center and ship-date option that passes lane, calendar, lead-time, SKU-presence, dock, and diversion filters. Each order may instead be explicitly unassigned. Assigned lines may be partially filled, but they cannot split across distribution centers.

2.2 Variables and objective

Let $x_c \in \{0,1\}$ select candidate c , $z_o \in \{0,1\}$ select the unassigned outcome for order o , $f_{osc} \in \mathbb{Z}_{\geq 0}$ denote fulfilled cases of SKU s under candidate c , and $u_{os} \in \mathbb{Z}_{\geq 0}$ denote unfulfilled cases. Additional bounded binary and continuous variables linearize the thresholded order penalty. The objective is

$$\text{maximize } J = \sum(o,s,c) v(o,s) f(o,s,c) - \sum(o) P(o) - \sum(c) C(c) x(c)$$

where v_{os} is merchandise value per fulfilled case, P_o is the active unmet-demand penalty, and C_c is shipping cost. To preserve confidentiality across experiments, we report **objective capture**, J divided by total requested merchandise value, rather than raw real-data currency totals.

2.3 Core constraints

The exact model enforces:

$$\sum(c \in C(o)) x(c) + z(o) = 1; \sum(c \in C(o)) f(o,s,c) + u(o,s) = Q(o,s); 0 \leq f(o,s,c) \leq Q(o,s)x(c)$$

Group-linking equalities make all orders in a load choose the same option. For each distribution-center/SKU/date checkpoint, cumulative earlier fulfillment cannot exceed protected projected ATP. Resource rows constrain fixed dock use and any enabled case-dependent resources. A non-default assignment is valid only if its fill improves the protected default preview by both five percentage points and 100 cases, capped by demand. Exact pallet and loose-case variables are available when corresponding capacities are enabled. The complete linearization and assumptions are provided in [docs/mathematical_formulation.md](#) and [docs/assumptions.md](#).

3. Solver architecture

3.1 Classical hierarchy

The **default baseline** retains default routing and allocates feasible quantities. The **greedy baseline** orders whole assignment groups, evaluates eligible choices against current residual resources, commits the best deterministic incremental choice, and updates shared resources. **Polished greedy** fixes those assignments and lets the exact MILP re-optimize fulfillment and threshold penalties.

The **exact LNS** solver starts from polished greedy, selects assignment groups near resource or inventory conflicts, residualizes the inactive network, and solves a bounded MILP over the active neighborhood. It adapts neighborhood size under limits on orders, variables, runtime, and MIP gap. A candidate replaces the incumbent only after global validation and a strict objective improvement.

The full exact MILP is retained for small-instance certificates and backend comparison. SciPy's [milp](#) interface to the open-source HiGHS solver [14] is the default on Windows, macOS, and Linux. The same compiled objective, bounds, integrality vector, and sparse constraint matrix can optionally be submitted to Gurobi [15]. Gurobi is never imported or selected by default because installation and license availability vary by reviewer.

3.2 Hybrid neighborhood search

The hybrid solver changes only the local assignment proposal step:

1. select interacting assignment groups around the incumbent;
2. retain a bounded set of assignment plans for each group;
3. construct a QUBO with plan value, one-hot penalties, and pairwise resource-conflict surrogates;
4. sample with exact feasible enumeration, random search, simulated annealing, local QAOA, or an explicitly enabled IBM QPU;
5. measure raw one-hot and energy quality, then repair one-hot violations;

6. fix each retained assignment and solve exact fulfillment recourse;
7. merge the neighborhood into the incumbent and validate the entire solution; and
8. accept only a valid strict improvement.

This separation matters. QUBO penalties are useful for ranking local assignments, but they are not trusted as an exact representation of every business rule. Feasibility of the final plan is evidence for the recourse-and-validation pipeline, not proof that the raw sampler natively satisfied the constraints.

3.3 Strengthened quantum proposal engine

For group g with m_g choices, the gate-model implementation prepares a weight-one Dicke (W) state and uses an XY mixer that preserves one excitation. The final code uses a linear controlled-rotation W-state construction instead of a generic state-preparation instruction. It also changes the default mixer from a ring to the connected path

$$E_g = \{(0,1), (1,2), \dots, (m_g-2, m_g-1)\},$$

reducing logical mixer edges from m_g to m_g-1 when $m_g > 2$ while retaining reachability within the one-hot subspace. QAOA angles are optimized using the exact feasible-subspace simulator, cached by QUBO and depth, and reused across matched hardware mitigation variants. This prevents repeated local angle optimization from being counted as distinct quantum work.

Sampler evaluation now reports raw one-hot rate, exact optimum hit rate with a 95% Wilson interval, optimum hit conditional on one-hot feasibility, rate within 1% of the feasible energy span, best and mean normalized feasible energy gaps, and corresponding uniform-feasible null values. These additions make rare-hit claims testable. The archived IBM run predates the path-mixer and linear-W changes, so its hardware outcomes are not attributed to the strengthened circuit until a matched rerun is completed.

4. Experimental design

The full reviewed study contains 247 rows from 14 experiment families. The design includes a common six-method comparison; repeated real-size and synthetic-size scaling; candidate-universe, candidate-count, business-penalty, and QUBO-penalty sensitivity; inventory shocks; QUBO coefficient noise; a readout-error proxy; Pareto-pruning and batching ablations; sampler ablation; and a synthetic coordination control. A separate 26-row archive contains two classical controls, six local-statevector controls, and 18 IBM hardware trials.

Every experiment records configuration, data scope, runtime environment, source and problem hashes, method, objective decomposition, fill, reassignments, runtime, model or sampler diagnostics, and validator residuals. The validator independently recomputes assignment completeness, demand balance, group cohesion, candidate eligibility, inventory checkpoints, capacity, diversion thresholds, integrality, and the business objective. All methods are compared using this common evaluator.

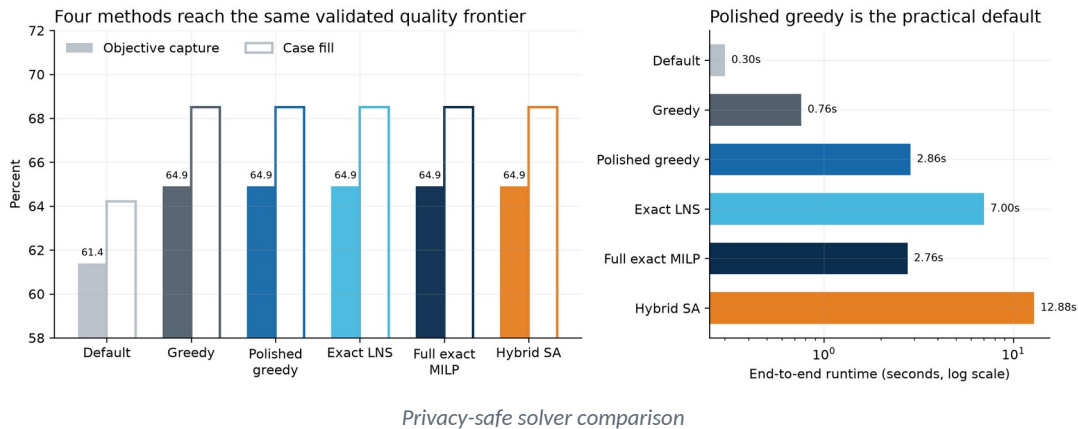
The real-data reports use normalized metrics and anonymized aggregates. Quantum hardware receives only a generated synthetic coordination instance. IBM trials used a 16-logical-qubit QUBO, 512 shots per trial, QAOA depths $p=1$ and $p=2$, three mitigation settings, and three repetitions on the 156-qubit [ibm_marrakesh](#) backend. Exact feasible enumeration, a uniform-feasible null, greedy, exact MILP, and local statevector QAOA are the controls. Because the hardware matrix has only three repetitions per cell, differences are descriptive rather than statistically conclusive.

5. Results

5.1 Common solver comparison

Method	Objective capture	Case fill	Reassigned orders	Runtime (s)	Interpretation
Default routing	61.39%	64.21%	0	0.30	Business baseline
Greedy	64.90%	68.50%	4	0.76	Fast constructive solution
Polished greedy	64.90%	68.50%	4	2.86	Recommended default
Exact LNS	64.90%	68.50%	4	7.00	Quality escalation
Full exact MILP	64.90%	68.50%	4	2.76	Zero-gap small-instance certificate
Hybrid simulated annealing	64.90%	68.50%	4	12.88	Experimental comparator

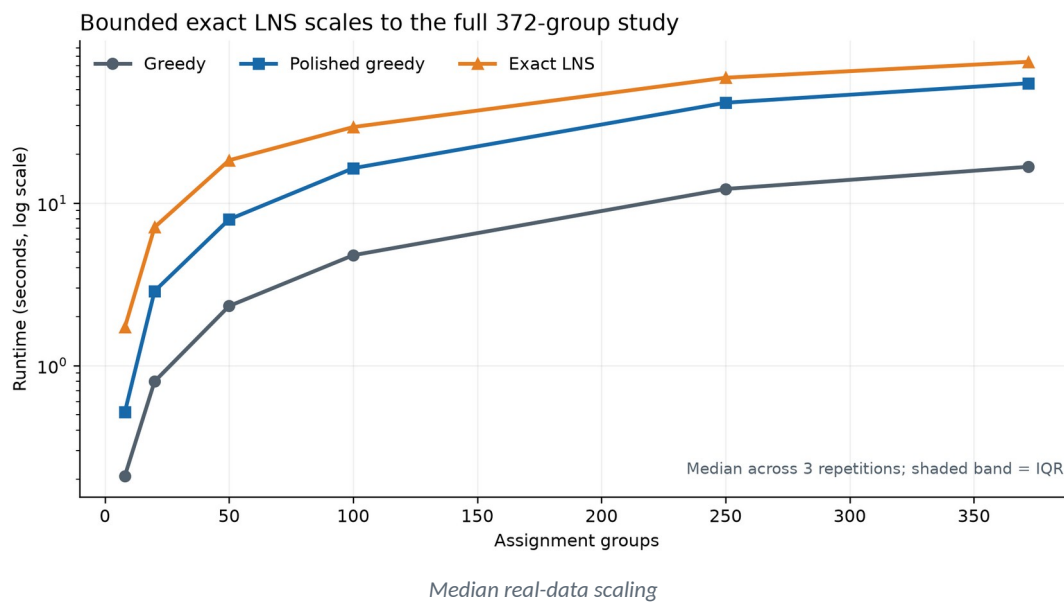
Greedy increased objective capture by 3.51 percentage points and case fill by 4.29 points relative to default routing. Exact fixed-assignment polishing added 119.44 raw objective units, only 0.004 percentage points of capture, and did not change fill. Exact LNS, full exact MILP, and hybrid search found no further accepted move on this subset. The result favors polished greedy as the routine default: it captures the verified quality plateau without paying the hybrid runtime. Full exact MILP remains valuable as a certificate at this size, not as the assumed full-scale production path.



5.2 Scaling

Repeated real subsets increased from 8 to 372 assignment groups and from 9 to 750 orders. Median greedy runtime rose from 0.21 to 16.74 seconds; polished greedy rose from 0.52 to 54.63 seconds; exact LNS rose from 1.72 to 74.03 seconds. The experimental hybrid was intentionally limited to at most 50 groups, where its median runtime was 28.70 seconds. Full exact MILP was intentionally limited to at most 20 groups.

At 100, 250, and 372 groups, exact LNS improved the polished incumbent by 22.98, 277.14, and 12.48 raw objective units respectively. Those gains are small relative to the normalized objective, but they show why a bounded escalation layer remains useful when conflicts grow. The operational policy should therefore allocate a runtime budget: run polished greedy for every planning cycle, then invoke exact LNS only for difficult windows or when the value of a certificate exceeds the added latency.



5.3 Coordination, robustness, and ablations

The synthetic coordination control was constructed so that independent greedy choices block a coordinated improvement. Greedy and polished greedy achieved 21.49% case fill and objective -2745.6. Exact LNS, full exact MILP, and the sampler-assisted hybrid reached 30.58% fill and objective -2548.4, a 197.2-unit improvement. Exact MILP needed 0.05 seconds, exact LNS 0.26 seconds, and the simulated-annealing hybrid 3.40 seconds. This is evidence that the *neighborhood architecture* can represent coordinated moves; it is not quantum advantage because classical exact methods solved the control faster.

Candidate-cap sensitivity showed the meaningful quality change between one retained candidate and two; larger caps were flat on the tested subset. Business penalty scaling changed the reported objective as expected but did not change routing or fill. All tested QUBO penalty combinations recovered the same synthetic optimum after recourse. Coefficient perturbations through 5% also preserved the final control solution.

Under aggregate inventory shocks of 10%, 25%, and 40%, every method remained valid. Re-optimized routing consistently exceeded fixed-routing recourse: at the 40% shock, case fill was 69.66% versus 67.21%, and objective capture was 65.16% versus 63.42%. This demonstrates the main operational value of rerouting under changing availability. Pareto pruning preserved quality and was slightly faster in this study, but it remains opt-in because isolated dominance is not a proof under shared resources. Conflict-based batching was slightly faster than random batching without changing the common-subset outcome.

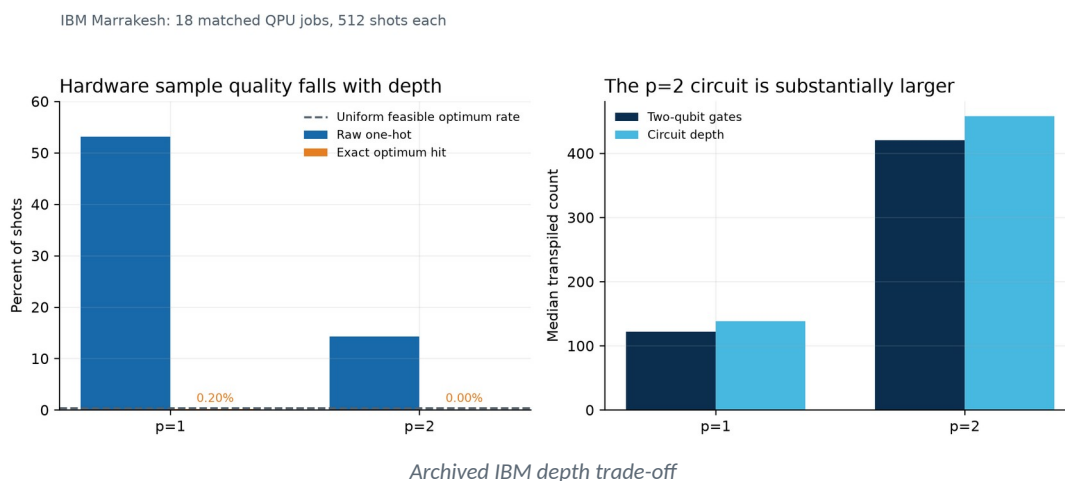
Sampler ablation is also cautionary. Exact enumeration, random sampling, simulated annealing, and local statevector QAOA all recovered the coordinated synthetic move after repair and recourse. The random sampler's native one-hot rate was approximately 0.26%, whereas the constraint-preserving samplers were one-hot by construction. Identical final decisions therefore do not imply identical raw sample quality; downstream recourse can mask large sampler differences.

6. Quantum hardware evidence

The IBM archive contains 18 QPU jobs and 512 shots per job. All QPU-derived final plans passed recourse and validation, and 16 of 18 recovered the best synthetic objective; the other two still improved on greedy. Across the 18 jobs, median native one-hot feasibility was 32.91%. Only a few shots hit the exact feasible QUBO optimum, as expected for a 256-state feasible space sampled 512 times per job.

Depth was the dominant signal. For $p=1$, the median transpiled circuit used 122 two-qubit gates and depth 138; median raw one-hot feasibility by mitigation cell ranged from 50.78% to 58.98%. For $p=2$, the median circuit used 420 two-qubit gates and depth 458; cell medians fell to 12.11%–15.23%, and no $p=2$ mitigation cell had a nonzero median exact-optimum hit rate. The deeper ideal statevector distribution had a better mean normalized gap than $p=1$ (0.283 versus 0.351 in matched 8,192-shot controls), but that theoretical improvement did not survive the hardware depth increase.

For $p=1$, measurement twirling plus dynamical decoupling had the largest observed median exact-hit rate, 0.3906%, while the unmitigated baseline had the strongest median one-hot feasibility, 58.98%. With only three trials per cell, neither difference supports a general mitigation claim. A uniformly sampled feasible state hits the unique optimum with probability $1/256=0.3906\%$ and has a mean normalized gap of 40.15%; this null context prevents overinterpreting a rare exact hit.



The correct conclusion is that hardware is safely integrated but not competitive. The QPU did not improve solution quality or wall-clock time over exact MILP on the control. The experiment identified the next engineering target—two-qubit depth—while the exact recourse layer prevented noisy samples from becoming invalid plans.

7. Discussion: advantages of the hybrid design

The hybrid solver has five defensible strengths.

First, **feasibility is solver-independent**. A separate validator recomputes all business constraints and objective terms. This catches adapter, heuristic, QUBO, and post-processing errors rather than trusting a solver status alone.

Second, **the hierarchy matches operational budgets**. Greedy construction supplies a fast plan, exact recourse improves quantities, exact LNS spends additional time only on interacting decisions, and full MILP is reserved for certificates. The architecture is valuable even if the quantum sampler is never enabled.

Third, **quantum scope is bounded and privacy-safe**. A local QUBO represents only a small active neighborhood, and the hardware study uses generated data. The approach avoids encoding the complete operational network into a prohibitive number of qubits or sending restricted tables to an external processor.

Fourth, **samplers are interchangeable under one evaluator**. Exact, random, simulated, statevector, and QPU proposals are compared using the same QUBO, recourse model, validator, and metrics. This makes future quantum claims falsifiable against strong classical controls.

Fifth, **deployment does not depend on a license or operating system**. NumPy, pandas, SciPy/HiGHS, and PyYAML support the default workflow on Windows, macOS, and Linux. Gurobi, IBM Runtime, and CUDA scoring are isolated optional extras. The optional Gurobi experiment is worthwhile as a backend benchmark, but the final study should treat it as a follow-up because no matched licensed run is present in the archived evidence.

8. Limitations and future research

The real-data evidence comes from one supplied planning snapshot and selected nested subsets. It estimates algorithmic behavior, not future business impact. Candidate-DC authorization, enterprise working calendars, throughput maxima, customer-specific all-or-nothing rules, and commercial approval boundaries require owner confirmation. Full exact MILP was evaluated only at small scales, and all hardware claims come from a 16-qubit synthetic control. The archived manifests also record a dirty worktree because Jupyter output autosaves changed a tracked notebook during the run; the final provenance implementation hashes normalized code-cell source separately, but old evidence is not retrospectively relabeled.

The following research directions are prioritized:

1. **Rerun the strengthened circuits.** Compare ring versus path mixers and generic versus linear W-state preparation on the same backend calibration, shots, seeds, and mitigation matrix. Report two-qubit depth and Wilson intervals before solution-level metrics.
2. **Adaptive depth and stopping.** Use shallow $p=1$ as the hardware baseline, stop a variant whose one-hot interval or normalized gap is dominated, and increase depth only when compiled resources remain within a calibrated error budget.
3. **Warm starts and parameter transfer.** Transfer angles across related rolling neighborhoods, evaluate warm-start QAOA, and separate classical optimization, queueing, QPU execution, and end-to-end time.
4. **Learning-guided neighborhoods.** Predict which inventory and dock conflicts are most likely to admit a coordinated gain, while retaining exact LNS as the control and validator as the acceptance gate.
5. **Stochastic and rolling-horizon DOM.** Model forecast error, cancellations, lead-time uncertainty, and repeated replanning with scenario or robust recourse rather than a single deterministic snapshot.
6. **Classical backend benchmark.** Run HiGHS and licensed Gurobi on the same compiled models, time limits, machines, and seeds; compare incumbent quality, bounds, nodes, and wall time. Solver agreement should be checked by independent validation, not objective equality alone.
7. **Operational pilot.** Add an explicit DC authorization table, working-day calendar, capacity denominators, approval thresholds, and planner feedback. Measure accepted recommendations and realized service, not only offline objective value.

9. Conclusion

This work delivers a complete, portable, and independently validated DOM optimization pipeline. On the common real-data subset, polished greedy reached the same displayed quality as exact LNS and full exact MILP in 2.86 seconds, making it the recommended default. Exact LNS remains a useful bounded escalation and demonstrated coordinated gains at larger scales and on a synthetic greedy trap. Quantum sampling is integrated as a replaceable local proposal engine behind deterministic repair, exact recourse, and global validation. IBM hardware results revealed a strong depth-feasibility trade-off and no quantum speed or quality advantage. The main result is therefore not a claim of quantum superiority; it is a safe experimental architecture in which a future quantum improvement can be measured without weakening the operational plan.

References

- [1] H. Zhao, M. O. Ball, and M. Kotake, "Optimization-based available-to-promise with multi-stage resource availability," *Annals of Operations Research*, vol. 135, pp. 65–85, 2005. <https://doi.org/10.1007/s10479-005-6235-7>
- [2] F. T. S. Chan, S. H. Chung, and K. L. Choy, "Optimization of order fulfillment in distribution network problems," *Journal of Intelligent Manufacturing*, vol. 17, pp. 307–319, 2006. <https://doi.org/10.1007/s10845-005-0003-z>

- [3] M. Vázquez-Noguerol, P. Comesaña-Benavides, A. Poler, and J. Prado-Prado, "An optimisation approach for the e-fulfilment problem with order splitting and delivery time windows," *Central European Journal of Operations Research*, vol. 30, pp. 1369–1402, 2022. <https://doi.org/10.1007/s10100-021-00778-x>
- [4] P. Shaw, "Using constraint programming and local search methods to solve vehicle routing problems," in *Principles and Practice of Constraint Programming—CP98*, LNCS 1520, pp. 417–431, 1998. https://doi.org/10.1007/3-540-49481-2_30
- [5] M. Fischetti and A. Lodi, "Local branching," *Mathematical Programming*, vol. 98, pp. 23–47, 2003. <https://doi.org/10.1007/s10107-003-0395-5>
- [6] E. Danna, E. Rothberg, and C. Le Pape, "Exploring relaxation induced neighborhoods to improve MIP solutions," *Mathematical Programming*, vol. 102, pp. 71–90, 2005. <https://doi.org/10.1007/s10107-004-0518-7>
- [7] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," 2014. <https://arxiv.org/abs/1411.4028>
- [8] S. Hadfield et al., "From the Quantum Approximate Optimization Algorithm to a Quantum Alternating Operator Ansatz," *Algorithms*, vol. 12, no. 2, art. 34, 2019. <https://doi.org/10.3390/a12020034>
- [9] A. Bärttschi and S. Eidenbenz, "Deterministic preparation of Dicke states," in *Fundamentals of Computation Theory*, LNCS 11651, pp. 126–139, 2019. https://doi.org/10.1007/978-3-030-25027-0_9
- [10] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, "XY mixers: analytical and numerical results for the quantum alternating operator ansatz," *Physical Review A*, vol. 101, art. 012320, 2020. <https://doi.org/10.1103/PhysRevA.101.012320>
- [11] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum Approximate Optimization Algorithm: performance, mechanism, and implementation on near-term devices," *Physical Review X*, vol. 10, art. 021067, 2020. <https://doi.org/10.1103/PhysRevX.10.021067>
- [12] D. J. Egger, J. Mareček, and S. Woerner, "Warm-starting quantum optimization," *Quantum*, vol. 5, art. 479, 2021. <https://doi.org/10.22331/q-2021-06-17-479>
- [13] N. Tomesh, Z. H. Saleem, and M. Suchara, "Quantum local search with the quantum alternating operator ansatz," *Quantum*, vol. 6, art. 781, 2022. <https://doi.org/10.22331/q-2022-08-22-781>
- [14] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Mathematical Programming Computation*, vol. 10, pp. 119–142, 2018; HiGHS project documentation. <https://highs.dev/>
- [15] Gurobi Optimization, LLC, *Gurobi Optimizer Reference Manual*, 2026. <https://docs.gurobi.com/projects/optimizer/en/current/>
- [16] IBM Quantum, *Qiskit Runtime documentation*, 2026. <https://quantum.cloud.ibm.com/docs/en/guides>